

Hostel Management System

High-Level Design | Version 1.0 | 14 September 2026

Document status: Current implementation and target architecture

Executive Summary

The Hostel Management System is a TypeScript and Node.js microservice platform for users, roles, rooms, bookings, wallet-backed payments, and payment notifications. HTTP APIs handle commands and queries while Kafka carries asynchronous business events.

Booking creation uses a transactional outbox so a committed booking remains recoverable if Kafka is unavailable. Payment and notification processing are independently scalable consumers. Prometheus, Grafana, Kafka-Monitor, Redis, and Blackbox Exporter provide the operational foundation.

Production readiness focus: idempotent consumers, atomic wallet debits, outbox row claiming, bounded retries with DLQs, event schema validation, and clear payment-data ownership.

Goals and Non-Goals

Goals

- Provide secure APIs for identity, access control, inventory, and bookings.
- Accept bookings without synchronously waiting for payment or email.
- Preserve committed bookings during temporary Kafka outages.
- Scale payment and notification independently.
- Expose service, Kafka, cache, and business-flow telemetry.

Non-goals

- UI or mobile-client design.
- Cloud-provider or Kubernetes manifests.
- Implementation of Kafka or MSSQL themselves.

- An exactly-once processing guarantee.

Component Overview

Component	Responsibility	Runtime
Booking-Service	Authentication, RBAC, inventory, bookings, outbox, payment-result consumer	Node.js/TypeScript, :3060
Payment-Service	Wallets, payments, booking-event consumer, payment-result producer	Node.js/TypeScript, :3061
Notification-Service	Payment-result consumer and SMTP email delivery	Node.js/TypeScript, :3062
Kafka-Monitor	Consumer-lag metrics for configured topic/group pairs	Node.js/TypeScript, :3070
Platform services	Kafka, MSSQL, Redis, Prometheus, Grafana, Blackbox Exporter	Environment-defined / 6379 / 9090 / 3000 / 9115

Core Booking and Payment Flow

1. Client submits a booking request to Booking-Service.
2. Booking-Service validates dates, room availability, user, overlap, and amount.
3. One MSSQL transaction writes the booking, room mapping, and pending outbox event.
4. The outbox worker publishes `booking.created`.
5. Payment-Service validates the wallet and publishes `payment.result`.
6. Booking-Service changes the booking to `CONFIRMED` or `PAYMENT_FAILED`.
7. Notification-Service sends the corresponding email.

Event Contracts

Topic	Producer	Consumers	Key
booking.created	Booking-Service outbox	Payment-Service, payment-group	booking ID
payment.result	Payment-Service	Booking-Service, booking-group ; Notification-Service, notification-group	booking ID

The effective delivery model is **at least once**. Consumers must tolerate redelivery and use `bookingId` as an idempotency key. Event versioning and schema validation are required before production scale.

Data and Ownership

Domain	Current owner	Note
Users, roles, permissions	Booking-Service	Identity and access control
Room types and rooms	Booking-Service	Inventory and cached room-type reads
Bookings and mappings	Booking-Service	Booking lifecycle and outbox
Wallets	Payment-Service	Wallet operations
Payments	Shared/current implementation	Must be assigned to Payment-Service or formalized as a shared contract

Redis uses cache-aside reads with a five-minute TTL for room types. MSSQL remains authoritative. Price-sensitive booking logic should read authoritative pricing within the booking transaction or use versioned pricing.

Reliability and Security

- Add idempotent payment and booking-result consumers.
- Make wallet debit and payment transition atomic.
- Claim outbox rows before publishing from multiple workers.
- Bound retries and route poison messages to retry or DLQ topics.
- Protect Payment-Service routes with authentication and authorization.
- Use TLS/SASL for Kafka outside a trusted network.
- Keep secrets out of source control and redact sensitive logs.
- Add audit events for wallet debits, payment transitions, access changes, and cancellations.

Observability and Deployment

Prometheus scrapes Booking-Service, Payment-Service, Notification-Service, Kafka-Monitor, Redis Exporter, and Blackbox Exporter. Grafana presents dashboards for service health, Kafka lag, booking states, payment failures, outbox age, notification delivery, and cache health.

The current monitoring Compose stack starts Redis, exporters, Prometheus, and Grafana. Kafka and Node.js services run outside that Compose file. Liveness and readiness should be separate so dependency outages are visible without confusing process health with workload readiness.

Production Roadmap

1. Remove or feature-flag forced consumer test failures.
2. Implement atomic wallet/payment transactions.
3. Implement consumer idempotency and outbox row leases.
4. Define event schemas, retry limits, DLQs, and replay procedures.
5. Clarify payment database ownership.
6. Add readiness endpoints, alerts, audit logging, and sensitive-data redaction.

Repository Source Map

Concern	Source
Booking app and routes	Booking-Service/src/app.ts
Booking orchestration	Booking-Service/src/services/booking.service.ts
Outbox	Booking-Service/src/outbox/
Kafka integration	Booking-Service/src/kafka/ , Payment-Service/src/kafka/
Monitoring	Monitoring/prometheus/prometheus.yml , Monitoring/docker-compose.yml

Prepared from the current repository implementation. Use the Markdown companion for Confluence import and collaboration.